
Documentação de Projeto

para o sistema

Stream Sentry

Versão 1.5

Projeto de sistema elaborado pelo aluno Rafael Parreira Chequer e apresentado ao curso de Engenharia de Software da PUC Minas como parte do Trabalho de Conclusão de Curso (TCC) sob orientação dos professores Cleiton Silva Tavares, Danilo de Quadros Maia Filho, Leonardo Vilela Cardoso e Raphael Ramos Dias Costa.

14 de dezembro de 2025

Tabela de Conteúdo

Tabela de Conteúdo	2
Histórico de Revisões	2
1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelos de Usuários	2
2.3 Modelo de Casos de Uso e Histórias de Usuários	4
2.4 Diagrama de Sequência do Sistema e Contrato de Operações	6
3. Diagramas	10
3.1 Diagrama de Classes	10
3.2 Diagramas de Sequência	11
3.3 Diagramas de Comunicação	13
3.4 Arquitetura Lógica	15
3.5 Diagrama de Atividade	16
3.6 Diagrama de Componentes e Diagrama de Implantação	17
4. Projeto de Interface com Usuário	20
4.1 Interfaces Comuns a Todos os Atores	20
5. Modelos de Projeto	27
5.1 Modelo de Dados	28
5.2 Glossário	29
6. Plano de Testes	31
6.1 Plano de Teste de Aceitação	31
6.2 Plano de Teste de Integração	37
7. Cronograma e Processo de Implementação	38
7.1 Cronograma	38
7.2 Metodologia de Desenvolvimento	41
7.3 Ferramentas e Pipeline de CI/CD	41

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Rafael Parreira Chequer	17/09/2025	Criação inicial do documento para entrega da A3, contendo diagrama de casos de uso, modelos de usuários e projeto de tela.	1.0

Nome	Data	Razões para Mudança	Versão
Rafael Parreira Chequer	13/10/2025	Criação do diagrama de sequência de sistema, diagrama de classes, diagrama de sequência e diagrama de comunicação e adição das demais interfaces.	1.1
Rafael Parreira Chequer	02/11/2025	Adição da arquitetura lógica, diagrama de atividades, componentes e implantação. Inclusão do modelo de dados e do glossário.	1.2
Rafael Parreira Chequer	18/11/2025	Adição do cronograma.	1.3
Rafael Parreira Chequer	30/11/2025	Adição dos planos de teste, metodologia de desenvolvimento, ferramentas e pipeline de CI/CD.	1.4
Rafael Parreira Chequer	14/12/2025		1.5

1. Introdução

Este documento apresenta a análise de usuário, contexto, interface e interação do sistema Stream Sentry, uma ferramenta web *open-source* (licença MIT) desenvolvida em React para automação de testes *end-to-end* em aplicações de videoconferência baseadas em *WebRTC* ou *APIs* de videoconferência (ex.: Zoom SDK, Jitsi). A referência principal para a descrição do problema, domínio e requisitos é o Documento de Visão (anexo a este documento), que detalha o escopo, as funcionalidades, os usuários-alvo (desenvolvedores React, engenheiros de QA (*Quality Assurance*), equipes ágeis e pesquisadores acadêmicos) e as restrições do sistema.

2. Modelos de Usuário e Requisitos

Esta seção apresenta os modelos de usuário e os requisitos funcionais do sistema Stream Sentry, com base nas necessidades identificadas no Documento de Visão. São descritos os atores envolvidos, personas representativas, casos de uso, histórias de usuário, diagramas de interação e contratos de operações, garantindo alinhamento entre as expectativas dos usuários e as funcionalidades do sistema. O objetivo é fornecer uma base clara e estruturada para o desenvolvimento, priorizando usabilidade, automação e extensibilidade em cenários de teste de videoconferência.

2.1 Descrição de Atores

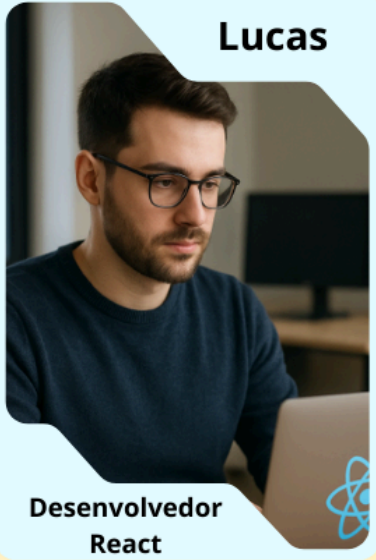
Os atores que interagem com o sistema Stream Sentry são definidos com base nos usuários-alvo descritos no Documento de Visão. Abaixo está a descrição de cada ator:

- **Desenvolvedor React:** Profissional que desenvolve aplicações de videoconferência em React, utilizando *WebRTC* ou *APIs* como Zoom SDK ou Jitsi. Este ator utiliza o Stream Sentry para automatizar testes *end-to-end*, reduzindo o tempo e o esforço em validações manuais.
- **Engenheiro de QA:** Responsável por garantir a qualidade de aplicações de videoconferência, utilizando o sistema para executar testes automatizados e analisar relatórios detalhados com métricas como latência, falhas e cobertura de testes.
- **Pesquisador Acadêmico:** Estudante ou professor que investiga automação de testes multimídia, utilizando o **Stream Sentry** como uma ferramenta *open-source* para experimentação e análise.
- **Equipe Ágil:** Grupo de profissionais (desenvolvedores, testadores e gerentes) que utilizam o sistema em fluxos de trabalho ágeis para integrar testes automatizados e relatórios em seus processos.

2.2 Modelos de Usuários

Para representar os usuários do sistema, foram desenvolvidas personas que refletem as características, necessidades e objetivos dos atores identificados. As personas são baseadas nas necessidades levantadas no Documento de Visão e ajudam a guiar o design do sistema.

Persona 1:



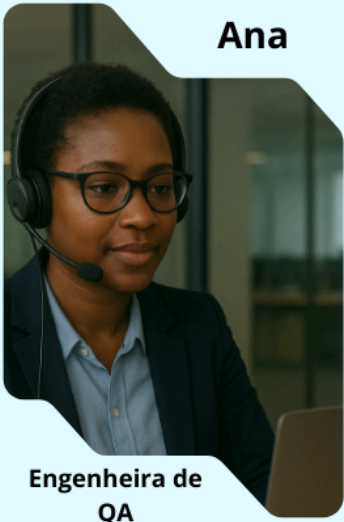
Lucas

**Desenvolvedor
React**

- **Idade:** 28 anos
- **Formação:** Bacharel em Ciência da Computação
- **Contexto:** Trabalha em startup desenvolvendo aplicações WebRTC
- **Objetivos:** Automatizar testes end-to-end para estabilidade e redução de bugs
- **Necessidades:** Interface simples, relatórios JSON/CSV, suporte a WebRTC
- **Frustrações:** Testes manuais demorados, ferramentas pagas caras
- **Cenário de Uso:** Configura teste com 5 usuários virtuais no StreamSentry e exporta relatório JSON

Figura 1: Lucas Desenvolvedor React

Persona 2:



Ana

Engenheira de QA

- **Idade:** 32 anos
- **Formação:** Engenharia de Software
- **Contexto:** Valida qualidade de áudio/vídeo em APIs como Jitsi
- **Objetivos:** Executar testes automatizados com métricas de latência e falhas
- **Necessidades:** Relatórios detalhados, interface intuitiva, suporte a headless browsers
- **Frustrações:** Ferramentas como Selenium não otimizadas para WebRTC
- **Cenário de Uso:** Configura teste de estabilidade com 10 usuários e exporta relatório CSV

Figura 2: Ana Engenheira de QA**Persona 3:**



Carla

Pesquisadora Acadêmica

- **Idade:** 40 anos
- **Formação:** Doutora em Ciência da Computação
- **Contexto:** Pesquisa testes multimídia com ferramentas open-source
- **Objetivos:** Testar hipóteses com ferramenta flexível
- **Necessidades:** Código open-source (MIT), documentação, suporte a Zoom SDK
- **Frustrações:** Ferramentas proprietárias e falta de documentação
- **Cenário de Uso:** Configura teste com Jitsi, modifica código e documenta resultados

Figura 3: Carla Pesquisadora Acadêmica

Persona 4:



Felipe

Tech Lead

- **Idade:** 35 anos
- **Formação:** Engenharia de Software com MBA em Gestão Ágil
- **Contexto:** Tech Lead de startup de videoconferência, coordenando CI/CD
- **Objetivos:** Automatizar testes WebRTC, reduzir tempo de feedback de qualidade
- **Necessidades:** Integração com CircleCI, suporte Kubernetes, notificações Teams
- **Frustrações:** Testes falhando com codecs, configuração em containers complexa
- **Cenário de Uso:** Integra StreamSentry no pipeline e valida chamadas.

Figura 4: Lucas Tech Lead

2.3 Modelo de Casos de Uso e Histórias de Usuários

O diagrama de casos de uso do sistema Stream Sentry foi desenvolvido para representar as interações principais dos atores com o sistema, conforme as funcionalidades descritas no Documento de Visão. O diagrama está disponível na pasta Artefatos do repositório com o nome UseCaseDiagram.puml e é apresentado na Figura 1.

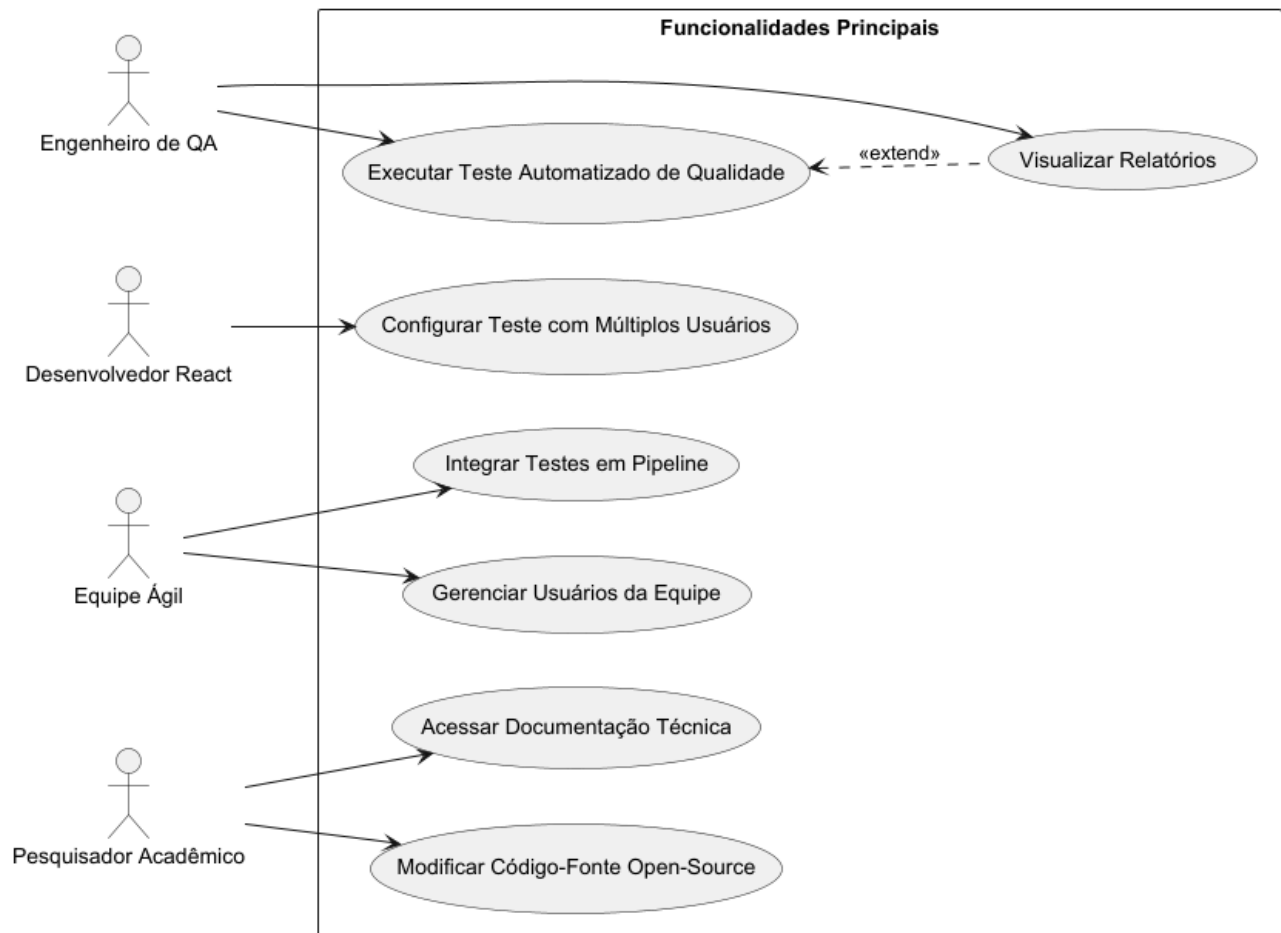


Figura 5: Diagrama de Casos de Uso do Stream Sentry

O diagrama ilustra os atores (Desenvolvedor React, Engenheiro de QA, Pesquisador Acadêmico, Equipe Ágil) e suas interações com os casos de uso principais: configurar testes, executar testes automatizados, visualizar relatórios e modificar código-fonte. Cada caso de uso reflete uma funcionalidade crítica ou importante identificada no Documento de Visão.

Abaixo está a lista de histórias de usuário, mapeadas com base nas necessidades e funcionalidades do sistema, utilizando identificadores únicos para referência:

- **US01:** Como Desenvolvedor React, eu quero configurar um teste com múltiplos usuários virtuais, para verificar a estabilidade da conexão em uma aplicação *WebRTC*, garantindo que a funcionalidade seja testada rapidamente.
- **US02:** Como Engenheiro de QA, eu quero executar testes automatizados de qualidade de áudio e vídeo, para identificar falhas de latência ou interrupções antes da entrega do software.
- **US03:** Como Engenheiro de QA, eu quero visualizar relatórios detalhados em formato JSON ou CSV, para compartilhar métricas de teste com a equipe e tomar decisões baseadas em dados.

- **US04:** Como Pesquisador Acadêmico, eu quero acessar o código-fonte do Stream Sentry, para modificá-lo e experimentar novos cenários de teste multimídia, atendendo aos meus objetivos de pesquisa.
- **US05:** Como membro de uma Equipe Ágil, eu quero configurar testes via uma interface web intuitiva, para integrar a automação ao fluxo de trabalho do time sem complicações.
- **US06:** Como Desenvolvedor React, eu quero que o sistema suporte *APIs* de videoconferência, para testar aplicações específicas sem necessidade de ferramentas adicionais.
- **US07:** Como Pesquisador Acadêmico, eu quero acessar documentação técnica completa, para entender a arquitetura do sistema e facilitar sua modificação para experimentos acadêmicos.

2.4 Diagrama de Sequência do Sistema e Contrato de Operações

Esta seção detalha a interação dinâmica entre o usuário e os componentes do sistema *Stream Sentry*. O fluxo foi dividido em três momentos lógicos: configuração, execução e exportação de resultados..

O fluxo de configuração, ilustrado na Figura 6, demonstra a sequência inicial onde o Desenvolvedor interage com o *Dashboard Web* para definir os parâmetros da simulação. A mensagem principal configurarTeste envia dados como quantidade de usuários e duração para o *Core*, que valida as informações e solicita ao Banco de Dados a persistência desses parâmetros, garantindo que o teste só seja considerado "configurado" após a confirmação de salvamento.

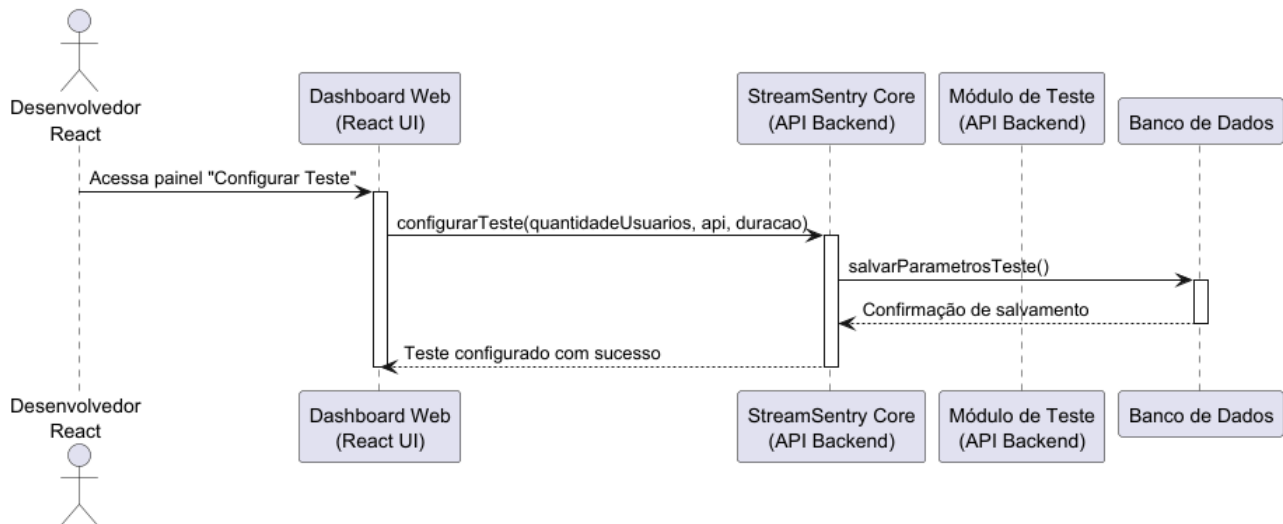


Figura 6: Diagrama de Sequência para Configuração de Testes

Na fase de execução, detalhada na Figura 7, o sistema opera em um ciclo de monitoramento ativo. O *Core* dispara o comando `executarTeste`, que ativa o *Módulo de Teste* (responsável por instanciar os clientes virtuais via Puppeteer). O diagrama evidencia a comunicação contínua onde o Módulo retorna métricas de latência e falhas, que são imediatamente enviadas pelo Core ao Banco de Dados para persistência, permitindo o acompanhamento em tempo real pela interface.

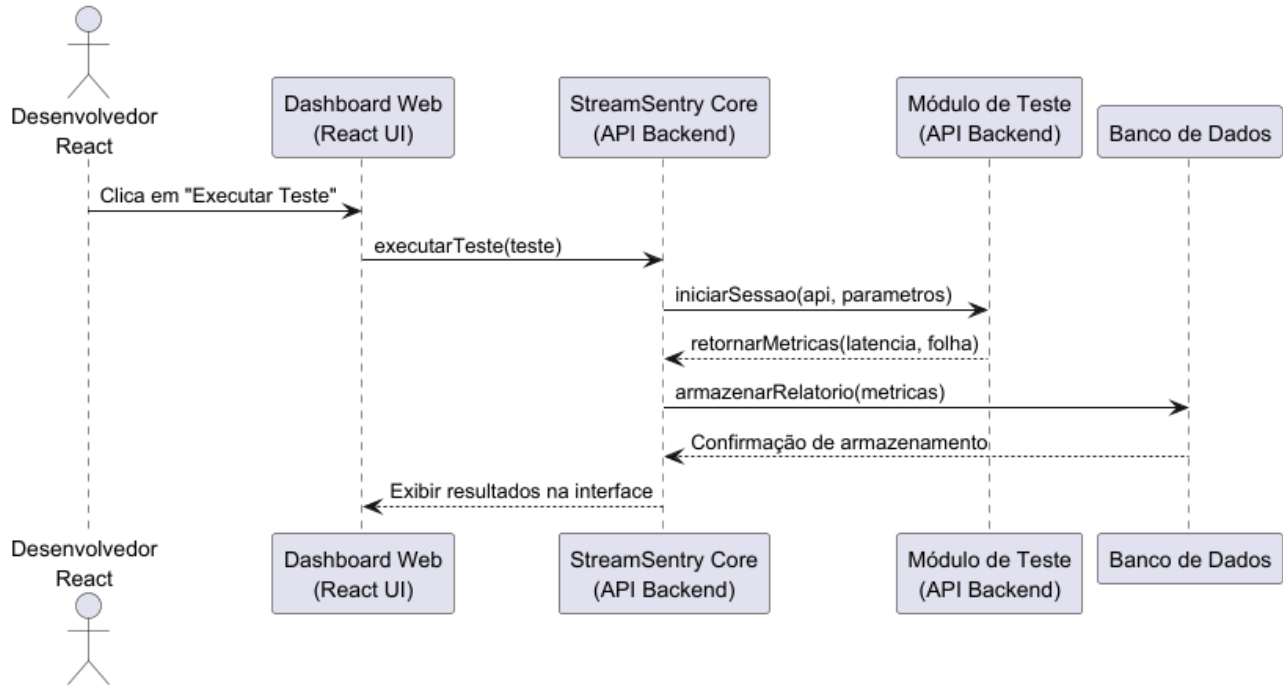


Figura 7: Diagrama de Sequência para Execução de Testes

Por fim, o processo de visualização e extração de dados é apresentado na Figura 8. O diagrama mostra o fluxo onde o usuário solicita visualizarRelatorio, especificando o formato desejado (JSON ou CSV). O *Stream Sentry Core* atua como intermediário, buscando o histórico consolidado no Banco de Dados e formatando a resposta para que o *Dashboard Web* possa exibir os gráficos ou iniciar o download do arquivo, encerrando o ciclo de uso.

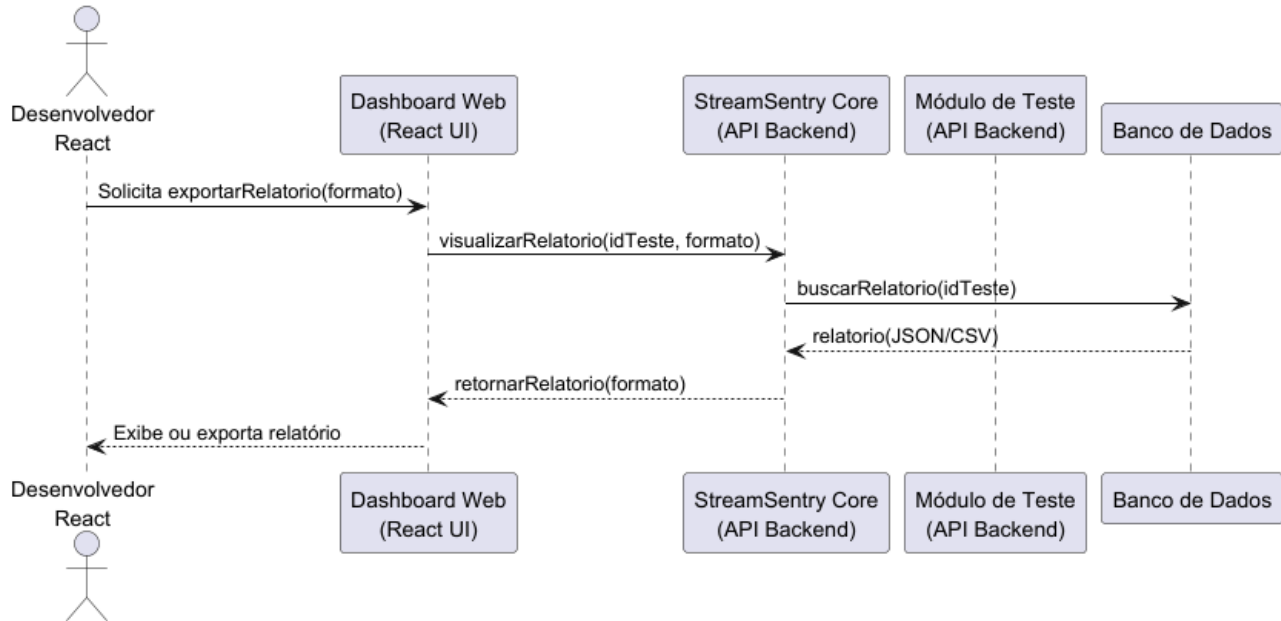


Figura 8: Diagrama de Sequência para Exportação de Relatório

A Tabela 1 formaliza o contrato da operação configurarTeste apresentada no diagrama acima. Ela especifica que, para o sucesso da operação, o usuário deve estar previamente autenticado e a API externa acessível. Como pós-condição, assegura-se que os parâmetros de entrada (número de usuários virtuais, tipo de API e duração) estejam armazenados de forma integridade no banco de dados, deixando o sistema em estado de prontidão para o início imediato da execução.

Contrato	Configurar Teste
Operação	configurarTeste(quantidadeUsuarios: int, api: string, duracao: int)
Referências cruzadas	US01, US05, US06
Pré-condições	- Usuário está autenticado. - API de videoconferência (ex.: WebRTC, Jitsi) está configurada e acessível.
Pós-condições	- Parâmetros do teste (número de usuários virtuais, tipo de API, duração) são salvos. - Sessão de teste é iniciada na API de videoconferência.

Tabela 1: Tabela de Configuração de Teste

O contrato operacional descrito na Tabela 2 define rigorosamente a função executarTeste. Ele estabelece como pré-requisito fundamental que o teste tenha sido configurado com parâmetros válidos e que a API de videoconferência alvo esteja ativa. As pós-condições garantem que a simulação de carga foi realizada e, crucialmente, que as métricas de desempenho (latência e falhas) foram coletadas e salvas, gerando a base para o relatório final.

Contrato	Executar Teste
Operação	executarTeste(idTeste: int)
Referências cruzadas	US02, US05
Pré-condições	- Teste está configurado com parâmetros válidos. - API de videoconferência está ativa.
Pós-condições	- Teste é executado com sucesso, e métricas (latência, falhas) são coletadas. - Relatório é gerado e armazenado para visualização.

Tabela 2: Tabela de Configuração de Teste

A Tabela 3 estabelece as regras para a operação visualizarRelatorio. O contrato exige que o teste tenha sido concluído com sucesso e que os dados já estejam disponíveis no banco. A saída esperada é a entrega do relatório detalhado, processado pelo backend para corresponder ao formato solicitado pelo usuário, garantindo a interoperabilidade dos dados gerados pelo *Stream Sentry* com outras ferramentas de análise.

Contrato	Visualizar Relatório
Operação	visualizarRelatorio(idTeste: int, formato: string)
Referências cruzadas	US03
Pré-condições	- Teste foi executado, e relatório está disponível. - Formato solicitado (JSON/CSV) é suportado.
Pós-condições	- Relatório é exibido na interface web ou exportado no formato solicitado.

Tabela 3: Tabela de Execução de Teste

3. Diagramas

A seção de Diagramas usa artefatos UML para apresentar uma visão integrada da estrutura, comportamento e arquitetura do Stream Sentry, cobrindo desde a modelagem estática até os fluxos dinâmicos e a implantação. Esses modelos sustentam a implementação, promovem a separação de responsabilidades e facilitam a compreensão, manutenção e escalabilidade do sistema em todos os seus níveis.

3.1 Diagrama de Classes

O Diagrama de Classes apresenta a arquitetura do Stream Sentry organizada em pacotes lógicos de Modelos, Core e Integração Externa, evidenciando o uso de padrões de projeto para garantir robustez e extensibilidade. A classe TestEngine atua como o núcleo orquestrador, utilizando a ProviderFactory para instanciar dinamicamente os drivers de conexão necessários. O padrão Strategy é aplicado através da interface IConferenceProvider, que define um contrato único para as implementações específicas (Jitsi, Zoom, WebRTC), permitindo a alternância entre APIs de vídeo sem gerar acoplamento rígido. Complementarmente, as classes MetricCollector e ReportGenerator isolam as responsabilidades de coleta de telemetria e exportação de dados, assegurando alta coesão entre a lógica de negócio e as entidades de domínio.

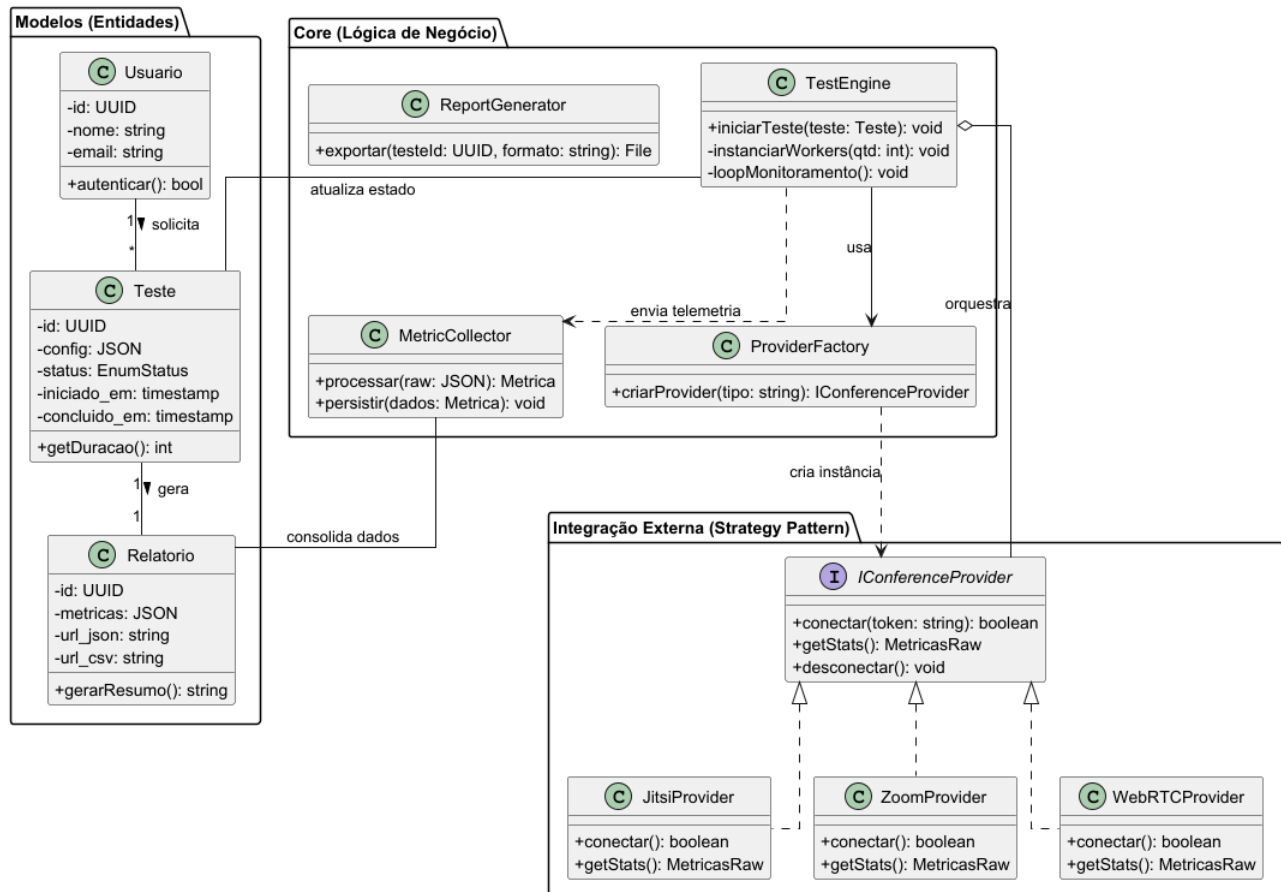


Figura 9: Diagrama de Classes do Stream Sentry

3.2 Diagramas de Sequência

O primeiro diagrama de sequência detalha o fluxo de Configuração e Validação, onde a integridade dos dados é assegurada antes de qualquer execução. O processo inicia-se quando o usuário define os parâmetros de teste na interface Web (React). Ao receber a requisição POST /configurar, o Core (Node.js) não atua apenas como persistência; ele executa uma validação lógica utilizando o padrão *Factory* para verificar a disponibilidade e compatibilidade do driver da API de videoconferência solicitada. Somente após essa verificação de integridade, o teste é salvo no Banco de Dados com status CONFIGURADO, retornando imediatamente um identificador único (UUID) ao frontend (HTTP 201), garantindo feedback instantâneo ao usuário.

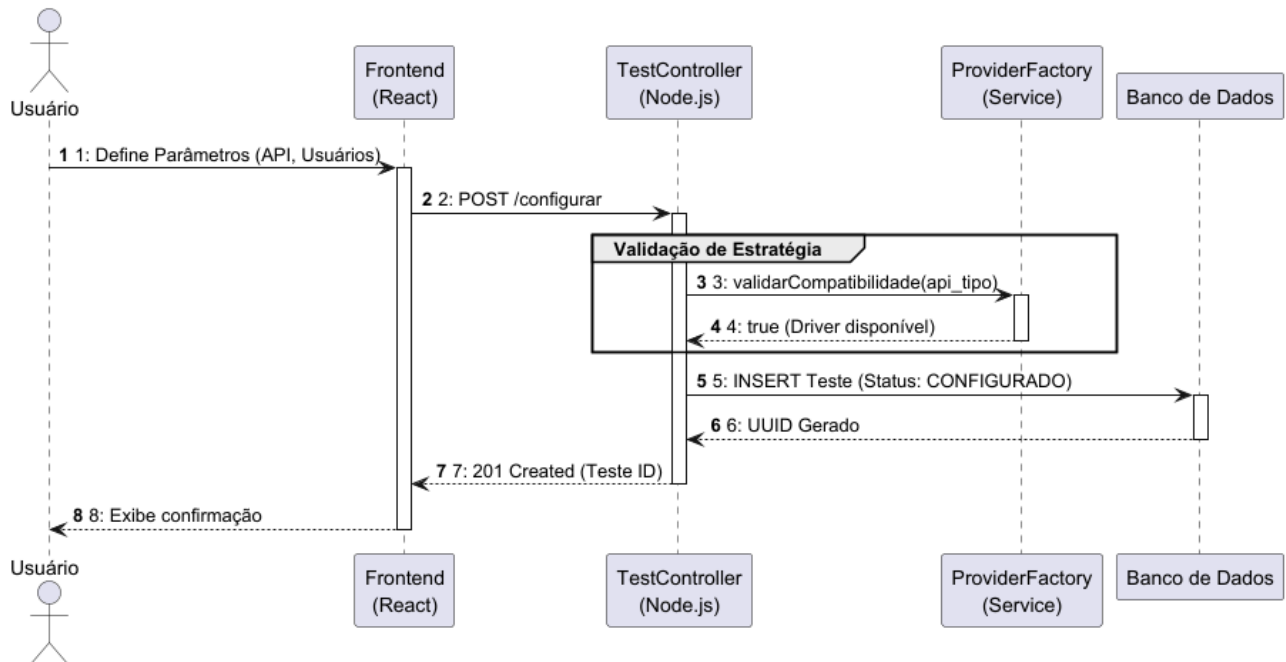


Figura 10: Diagrama de Sequência de Configuração e Validação

O segundo fluxo ilustra a Execução em Tempo Real, evidenciando a arquitetura assíncrona e orientada a eventos do sistema. Ao disparar o comando de início, o *TestEngine* orquestra a criação de instâncias isoladas (Workers) via Puppeteer, que estabelecem conexão WebRTC com a API externa. O diferencial arquitetural reside no loop de monitoramento: a cada ciclo de coleta de métricas, o sistema realiza operações paralelas de persistência no PostgreSQL e difusão de dados via WebSocket (`metric_update`). Isso elimina o bloqueio da requisição HTTP (observado em modelos síncronos) e permite que o Dashboard exiba gráficos de latência e falhas ao vivo enquanto o processamento pesado ocorre em segundo plano.

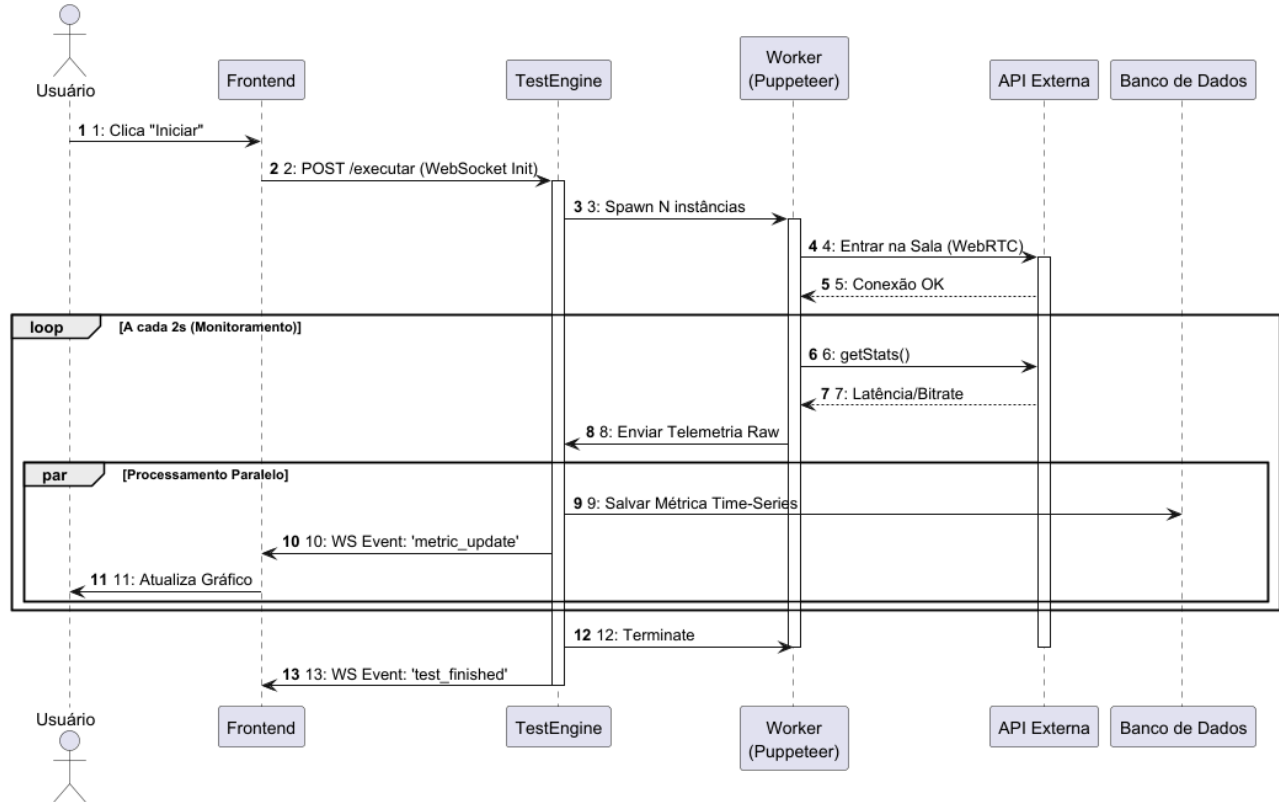


Figura 11: Diagrama de Sequência de Execução em Tempo Real

Por fim, o fluxo de Geração de Relatórios demonstra a separação de responsabilidades (SoC) no tratamento de dados históricos. Diferente de uma simples leitura direta, este processo é acionado sob demanda quando o usuário solicita a exportação. O *ReportGenerator* atua isoladamente, executando consultas de agregação complexas no banco de dados (cálculo de médias, desvio padrão e totais de erros) e utiliza estratégias de formatação específicas para converter o *dataset* processado em arquivos JSON ou CSV. O arquivo final é transmitido via *stream* para o usuário, assegurando que o processamento de grandes volumes de dados analíticos não impacte a performance dos testes que estejam ocorrendo simultaneamente.

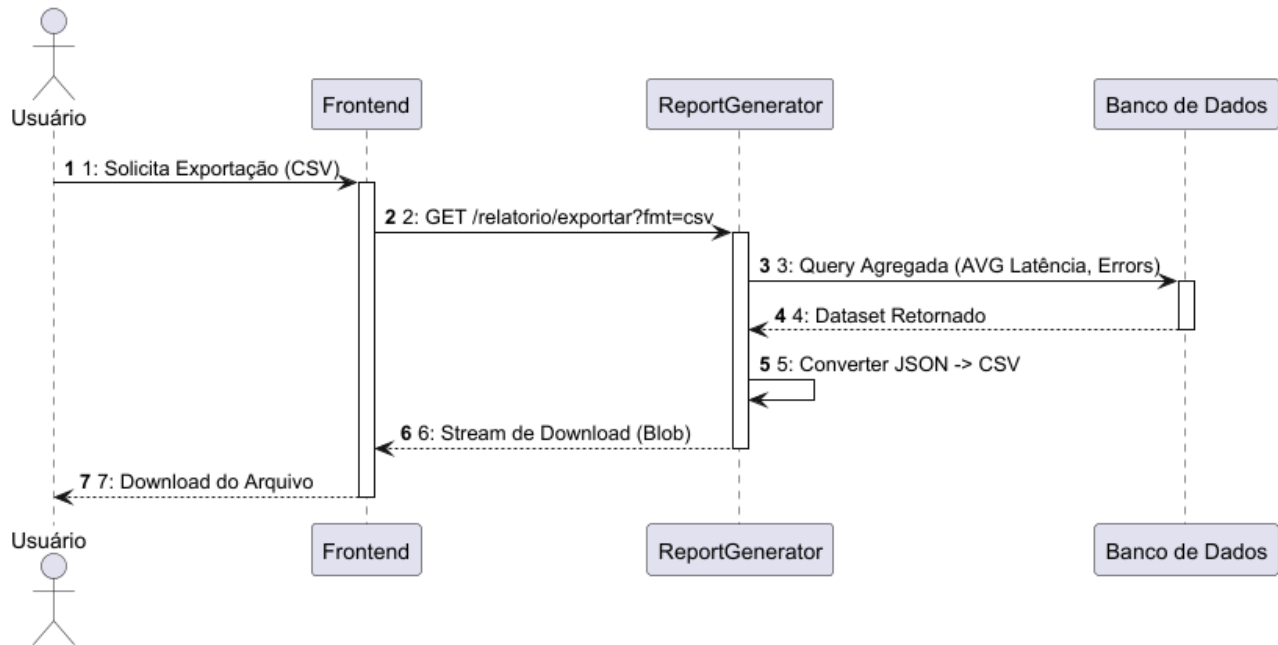


Figura 12: Diagrama de Sequência de Geração de Relatórios

3.3 Diagramas de Comunicação

O diagrama de comunicação de Execução e Telemetria ilustra o "caminho quente" (*hot path*) do sistema, focado na interatividade em tempo real. O fluxo inicia-se com o estabelecimento de um canal WebSocket persistente entre o Dashboard e o Core, permitindo comunicação bidirecional. O diagrama destaca a orquestração onde o Core instancia *Workers* isolados que interagem diretamente com a API de videoconferência. A ênfase recai sobre o loop de monitoramento contínuo (mensagens 6 a 11), demonstrando como as métricas coletadas são processadas paralelamente: sendo persistidas no Banco de Dados para histórico e, simultaneamente, transmitidas via eventos (*metric_update*) para atualização instantânea dos gráficos no frontend, evidenciando uma arquitetura

não-bloqueante.

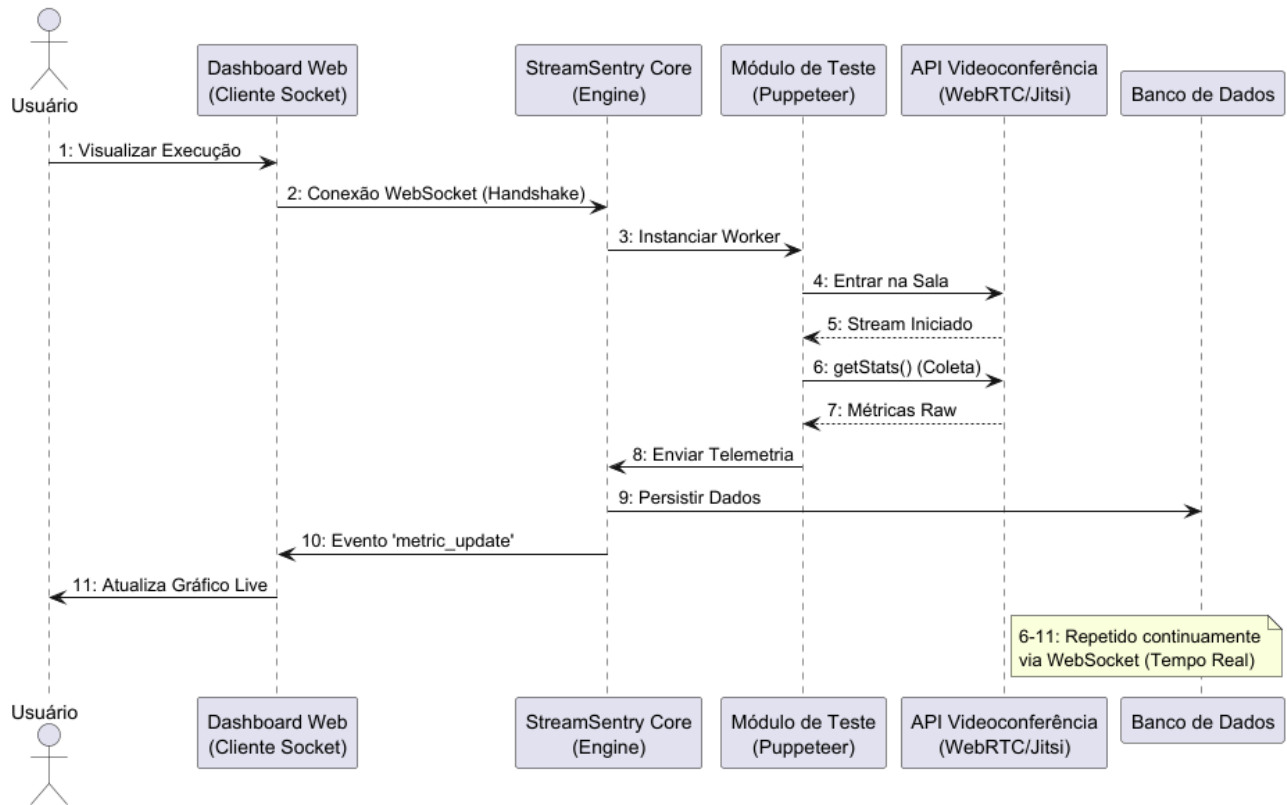


Figura 13: Diagrama de Comunicação de Execução e Telemetria

O diagrama de comunicação de Processamento de Relatórios detalha o "caminho frio" (*cold path*), focado na consolidação e transformação de dados históricos. O processo é disparado sob demanda pela interface de Relatórios, acionando o Core para executar consultas de agregação complexas no Banco de Dados, em vez de leituras simples. As mensagens evidenciam o desacoplamento arquitetural através do componente *Gerador de Arquivos*, que aplica uma estratégia de formatação (JSON ou CSV) sobre o *dataset* retornado antes de gerar o buffer de download. Essa segregação demonstra que a lógica de análise de dados é tratada de forma independente do núcleo de execução, garantindo a integridade e a performance do sistema.

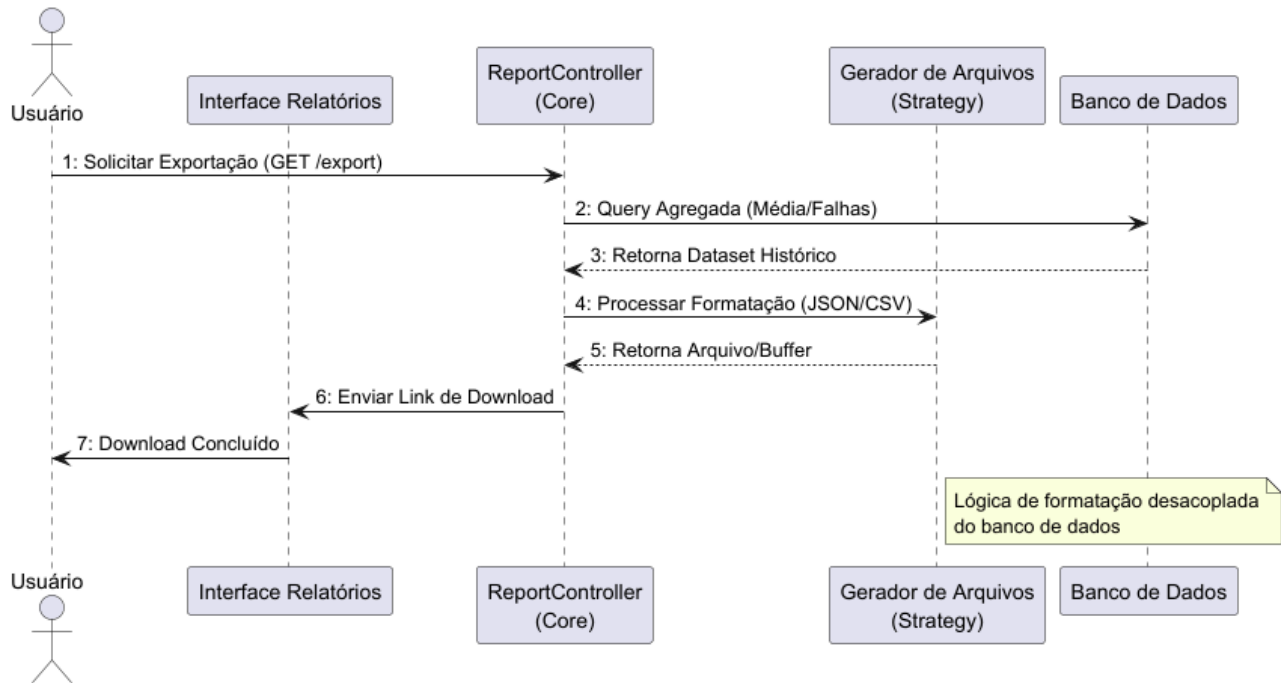


Figura 14: Diagrama de Processamento de Relatórios

3.4 Arquitetura Lógica

A arquitetura lógica do Stream Sentry é representada por um diagrama de pacotes UML que organiza o sistema em três camadas principais: Interface Web (React), Stream Sentry Core (Node.js) e Integração Externa. A camada de interface contém componentes de UI como Dashboard, Configurar Teste e Relatórios, consumindo APIs REST do backend. O núcleo do sistema é dividido em controladores (TestController, ReportController), serviços de negócio (TestEngine, MetricCollector, ReportGenerator) e persistência via PostgreSQL, garantindo separação clara de responsabilidades. A integração externa abrange Jitsi, Zoom SDK e WebRTC nativo, acessados via SDKs e WebSocket. Essa estrutura promove coesão, baixa acoplamento e escalabilidade, alinhada ao Documento de Visão e aos fluxos de dados dos diagramas de sequência e comunicação.

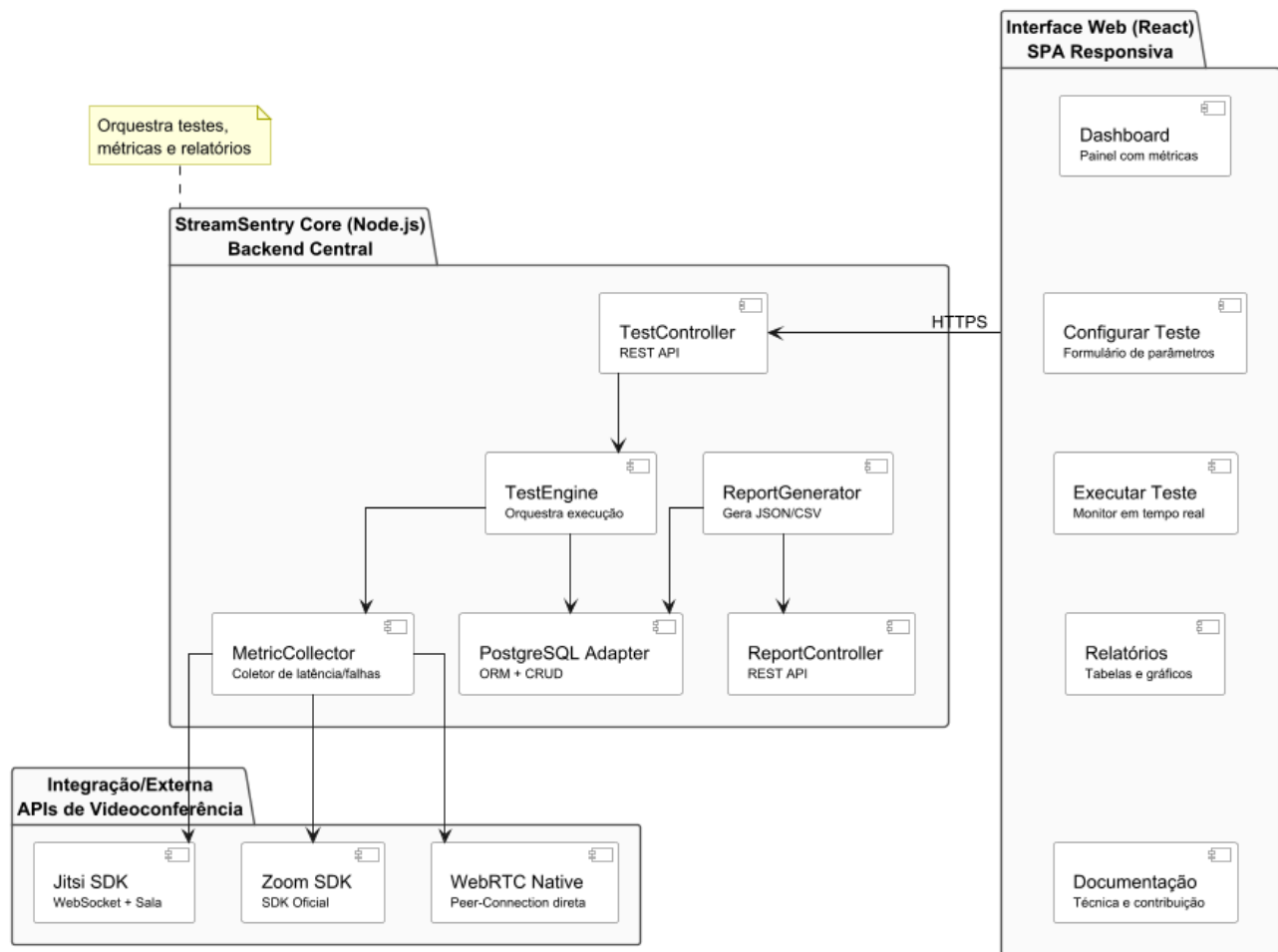
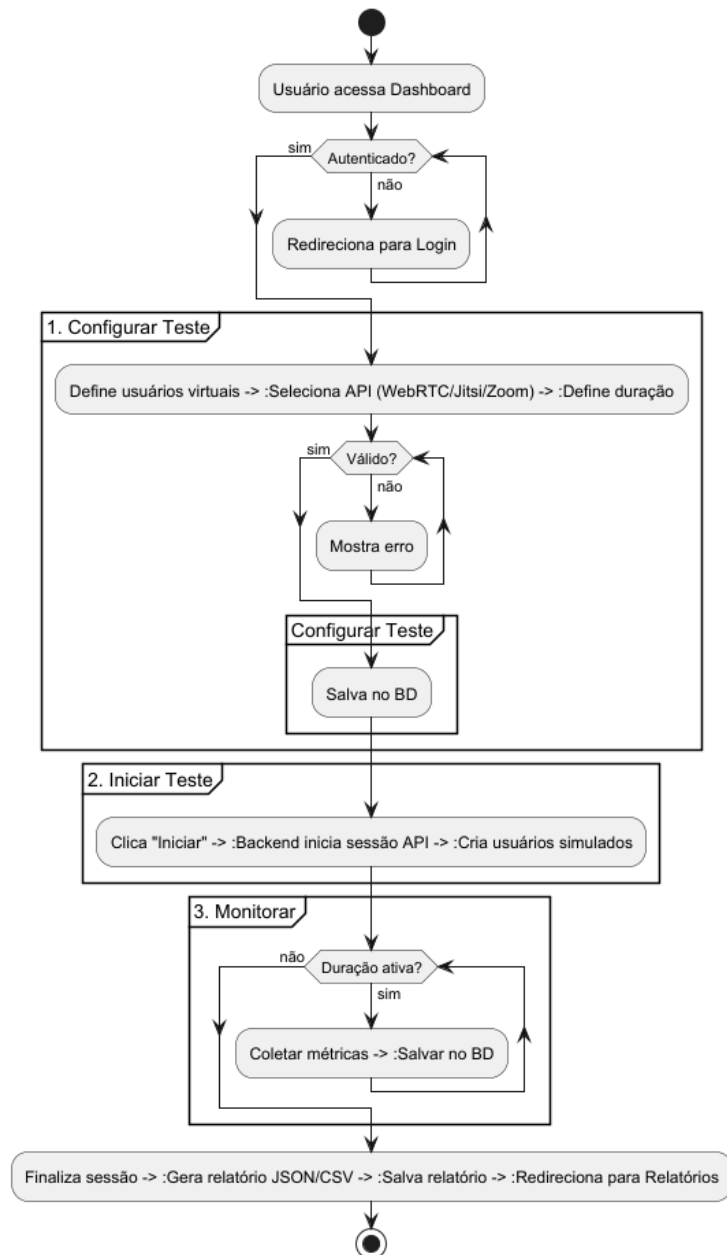


Figura 15: Diagrama de Arquitetura Lógica

3.5 Diagrama de Atividade

O Diagrama de Atividades (Figura 12) ilustra o fluxo "Configurar e Executar Teste": o usuário define os parâmetros (usuários, API, duração), que são validados e salvos no banco. A execução inicia, criando usuários virtuais e entrando em um loop de monitoramento de métricas. Ao término da duração, a sessão é finalizada, um relatório (JSON/CSV) é gerado e o usuário é redirecionado para a visualização.

**Figura 16: Diagrama de Atividade**

3.6 Diagrama de Componentes e Diagrama de Implantação

O Diagrama de Componentes do Stream Sentry (Figura 13) modela os módulos reutilizáveis em três camadas: Interface Web (React) com componentes de UI (Dashboard, Configurar Teste, Executar Teste, Relatórios e Documentação), Stream Sentry Core (Node.js) com controladores REST, serviços (Test Engine, Metric Collector, Report Generator) e persistência via PostgreSQL, e Integração Externa com Jitsi, Zoom SDK e WebRTC. As interfaces REST, WebSocket e SQL garantem o desacoplamento, permitindo comunicação clara e reutilização. Essa estrutura promove manutenibilidade e escalabilidade, alinhada ao Documento de Visão e aos fluxos de dados.

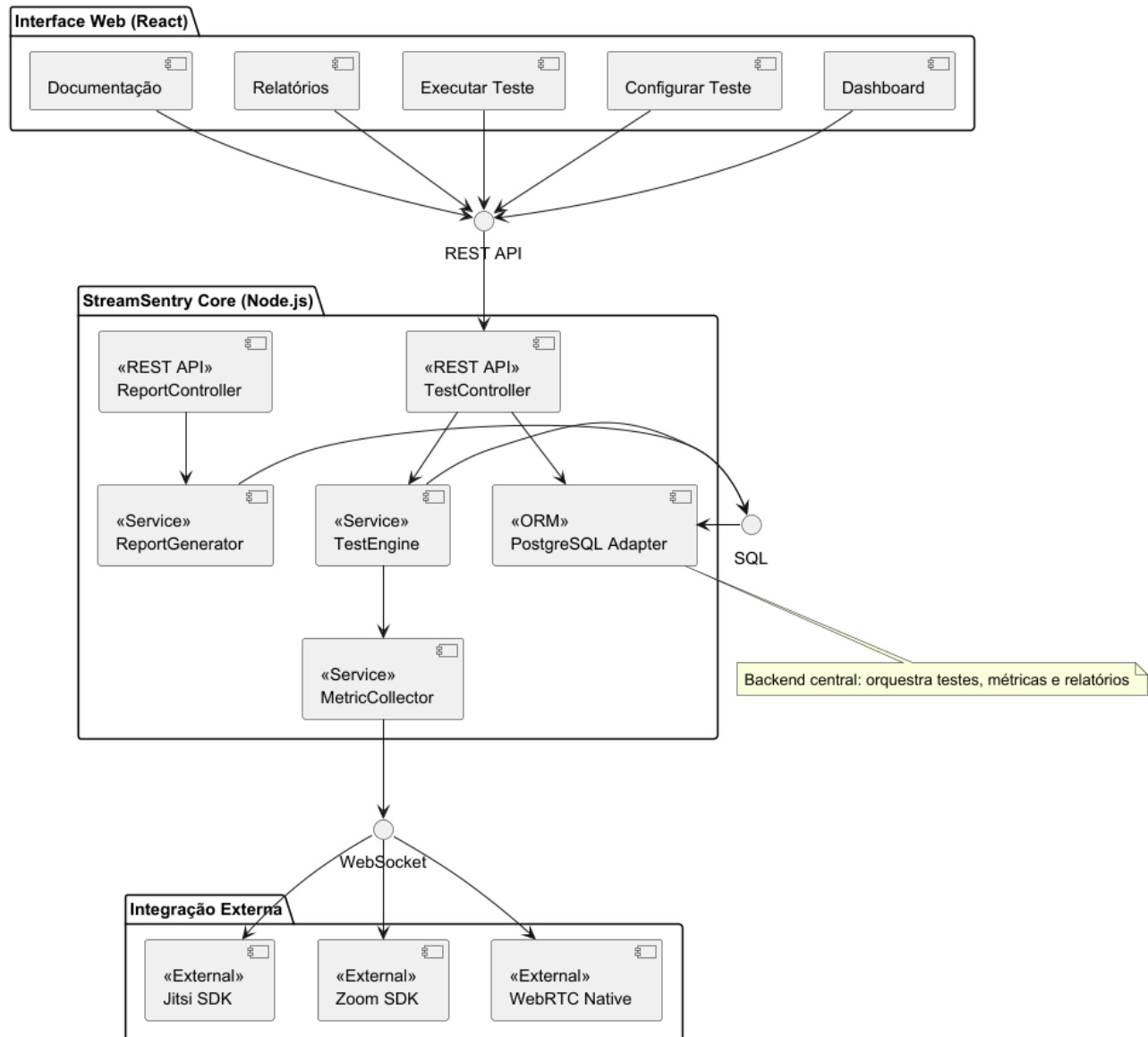


Figura 17: Diagrama de Componentes

O Diagrama de Implantação do Stream Sentry (Figura 14) define a infraestrutura física: o frontend (React SPA) é implantado em Vercel ou Netlify com build automático via GitHub, o backend (Node.js API) roda em Render ou AWS EC2 com container Docker e PM2 para alta disponibilidade, e o banco PostgreSQL é gerenciado (RDS ou Render). A comunicação ocorre via HTTPS (REST) entre frontend e backend, TCP 5432 (SQL) para persistência e WebSocket/HTTPS com APIs externas (Jitsi, Zoom, WebRTC). Essa arquitetura cloud-native garante escalabilidade, segurança e facilidade de deploy, atendendo aos requisitos de automação e integração do sistema.

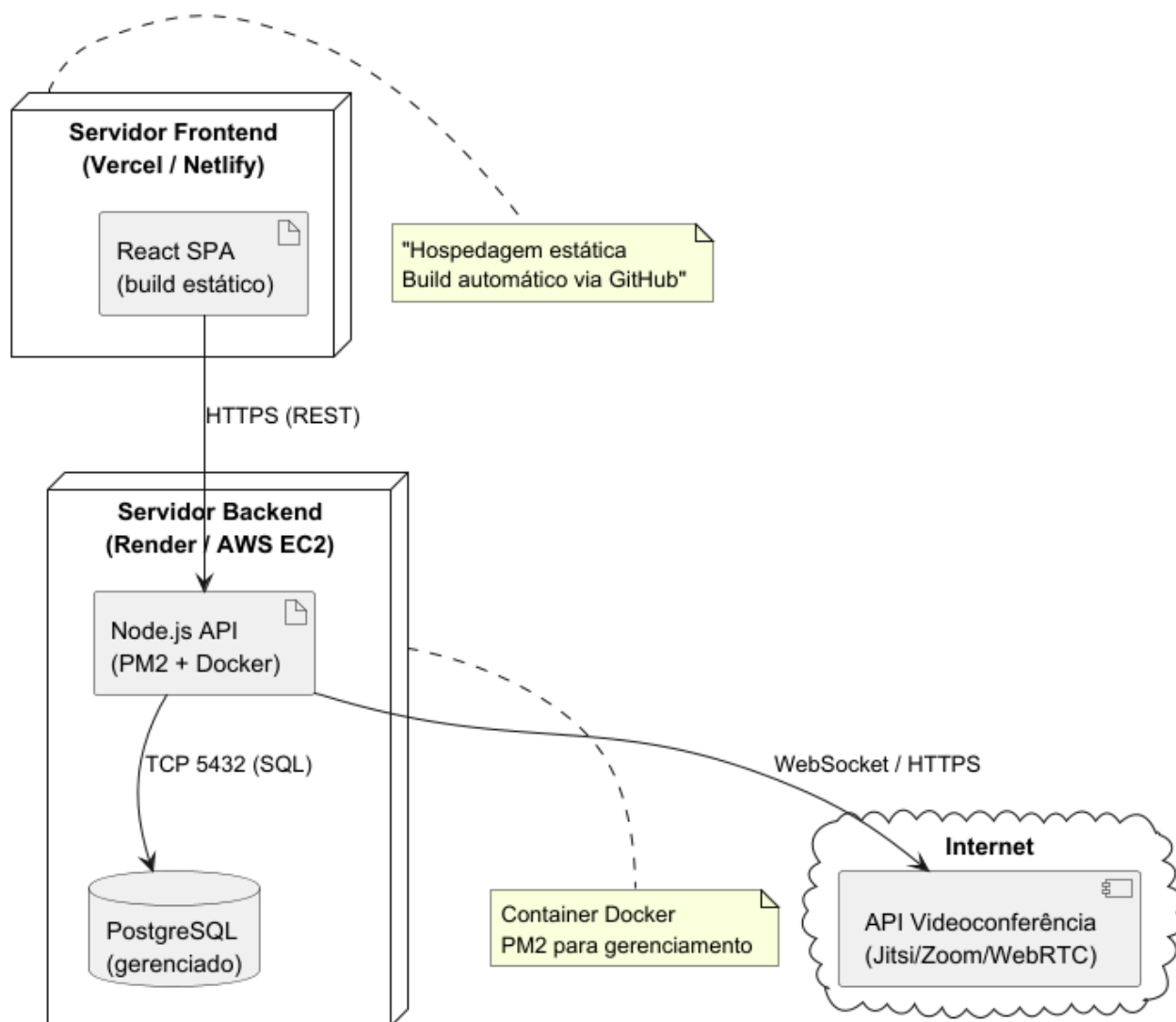


Figura 18: Diagrama de Implantação

4. Projeto de Interface com Usuário

O projeto de interface com usuário do Stream Sentry prioriza clareza técnica, usabilidade ágil e consistência visual, oferecendo uma experiência fluida para todos os atores (desenvolvedores, QA, pesquisadores e equipes ágeis). As interfaces comuns incluem um Dashboard central com visão geral de métricas, menu de navegação intuitivo e exportação de relatórios, enquanto telas específicas como Configurar Teste, Executar Teste, Relatórios, Documentação Técnica, Gerenciar Usuários, Histórico, Configurações Avançadas, Suporte e Login/Registro são projetadas com layouts responsivos, validação em tempo real, temas técnicos (modo escuro, terminais) e integração com GitHub. O design é minimalista, funcional e inspirado em ferramentas modernas (React Docs), garantindo acessibilidade, rastreabilidade e foco na automação de testes multimídia, com figuras ilustrativas que reforçam a coerência entre funcionalidade e experiência do usuário.

4.1 Interfaces Comuns a Todos os Atores

A tela inicial do sistema Stream Sentry oferece uma visão geral das operações e métricas de teste, com um menu superior contendo as opções Configurar Teste, Executar Teste, Relatórios e Documentação, um painel central exibindo gráficos e indicadores de desempenho como latência média, taxa de falhas e cobertura de testes, uma barra lateral com atalhos para configurações avançadas e acesso ao código-fonte, além de um botão de exportação para gerar relatórios em JSON ou CSV, tudo em um layout minimalista, responsivo e focado em clareza e usabilidade técnica.

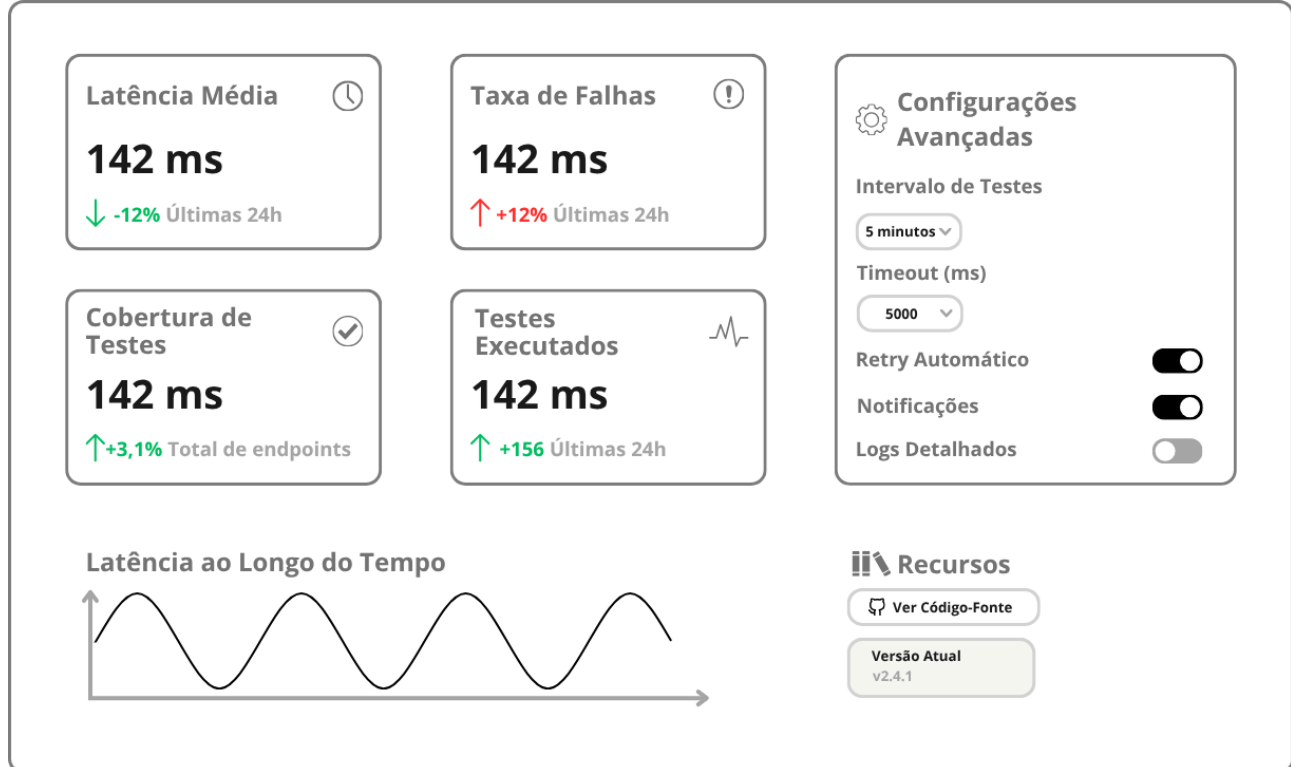


Figura 19: Tela Principal – Dashboard

Esta tela permite ao usuário definir os parâmetros de um novo teste automatizado, com campos para selecionar a quantidade de usuários virtuais (via controle deslizante), a API de videoconferência (WebRTC, Jitsi ou Zoom SDK) e a duração do teste (em minutos), exibindo indicadores de status como API conectada e usuários simulados prontos, além de botões “Salvar Configuração” e “Iniciar Teste” para execução direta, tudo em um design simples, com validação rápida de entradas e otimizado para fluxos ágeis.

A interface 'Configurar Teste' apresenta os seguintes elementos:

- Status da API:** Indica que a conexão foi estabelecida com o status 'Conectada'.
- Usuários Simulados:** Mostra 5/5 instâncias ativas.
- Parâmetros do Teste:** Seção para definir a quantidade de usuários virtuais, a API de videoconferência e a duração do teste.
 - Quantidade de Usuários Virtuais:** Controlado por um slider deslizante, atualmente configurado para 35.
 - API de Videoconferência:** Menu suspenso para selecionar a API para teste.
 - Duração do Teste:** Campo de entrada para definir a duração em minutos, atualmente configurado para 10.

Figura 20: Tela “Configurar Teste”

Esta tela é responsável por monitorar a execução de testes em tempo real, exibindo um painel de logs com informações de desempenho como latência, bitrate e falhas, além de um gráfico dinâmico que mostra variações ao longo do tempo, uma barra lateral de progresso indicando o status da execução e botões para pausar, finalizar ou gerar relatório, tudo em um visual com tema escuro inspirado em terminais técnicos, transmitindo precisão e controle.

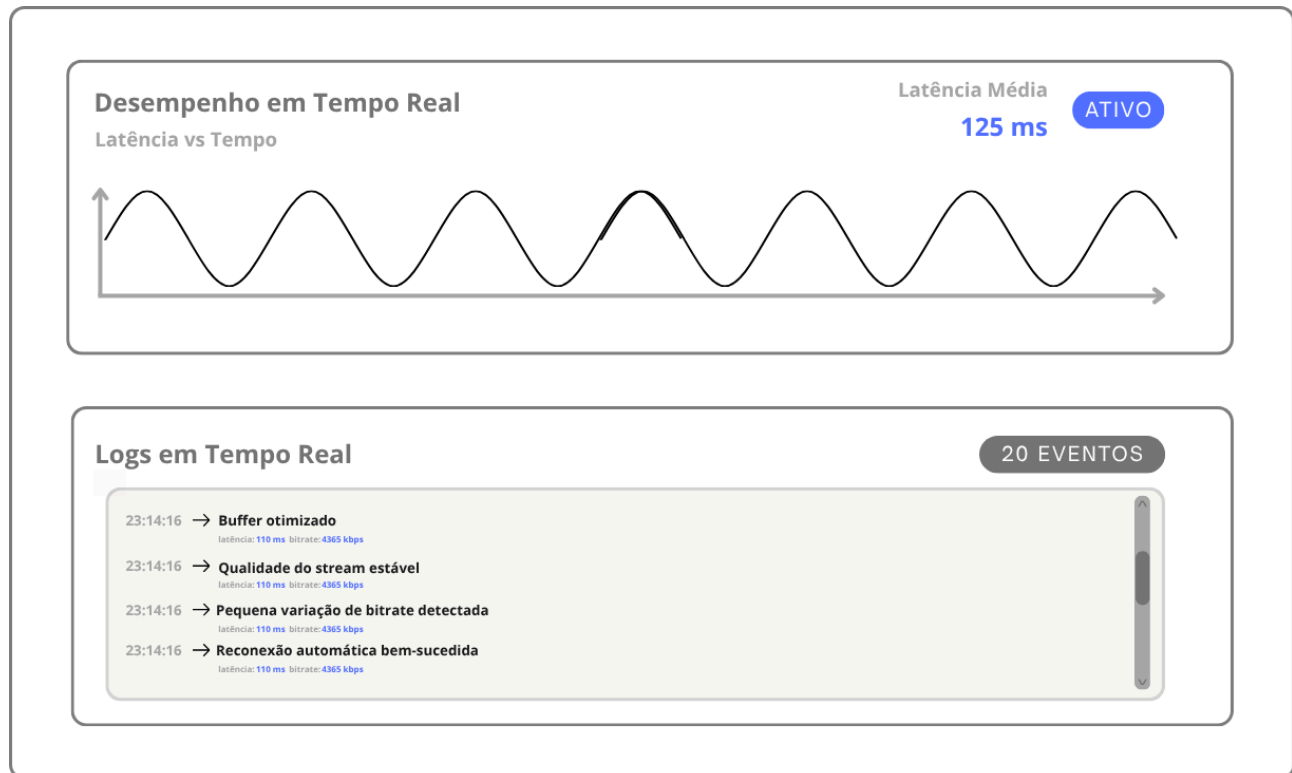


Figura 21: Tela “Executar Teste”

Esta tela centraliza o acesso aos resultados dos testes realizados, apresentando uma tabela com histórico de execuções incluindo API utilizada, número de usuários e status (Sucesso/Falha), exibindo ao selecionar um teste gráficos detalhados com métricas de latência, falhas e qualidade de vídeo/áudio, com opções para filtrar resultados por data, tipo de API ou status e exportar relatórios em JSON ou CSV, tudo em um layout de painel analítico com clareza visual e foco na interpretação rápida de dados.

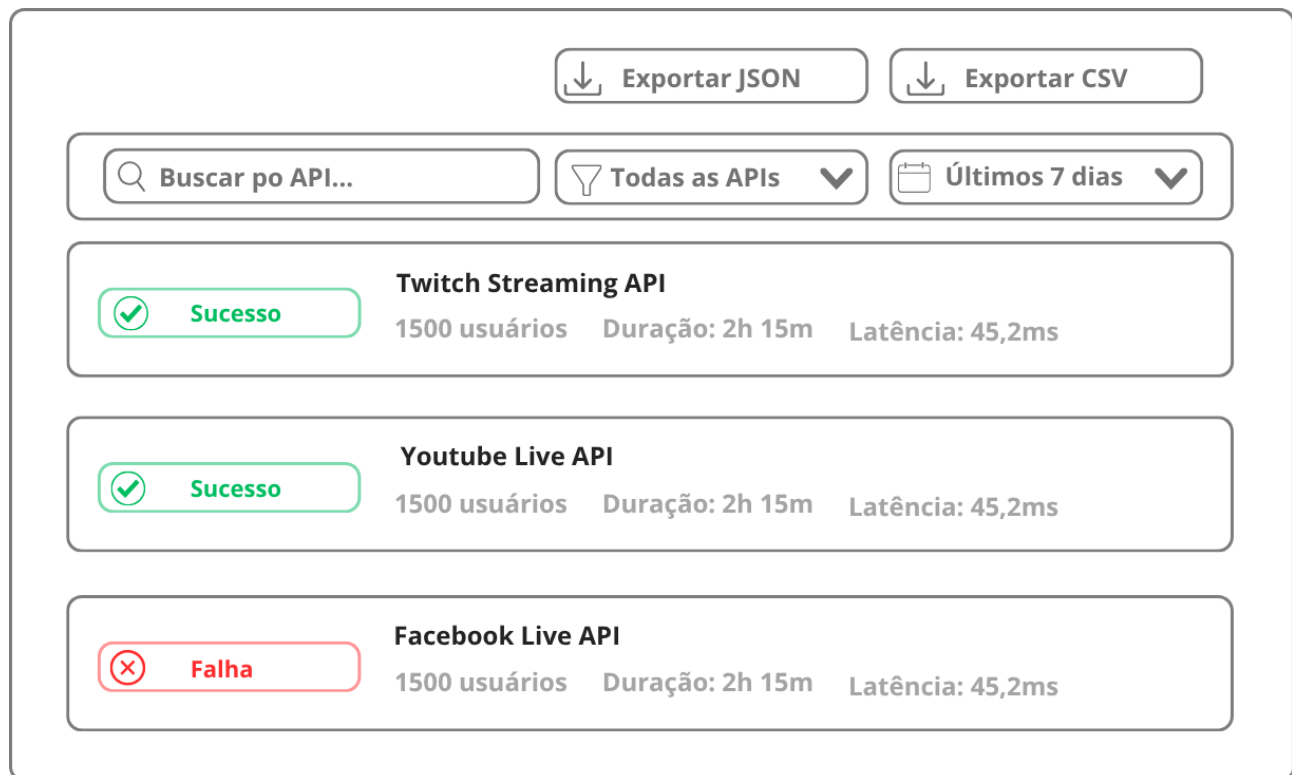


Figura 22: Tela “Relatórios”

Esta tela exibe uma linha do tempo vertical com todos os testes realizados, API usada, duração e status visual (ícones de sucesso/falha), com cada item contendo um link direto para o relatório detalhado, em um estilo inspirado em pipelines de integração contínua como o GitHub Actions, transmitindo automação e rastreabilidade.

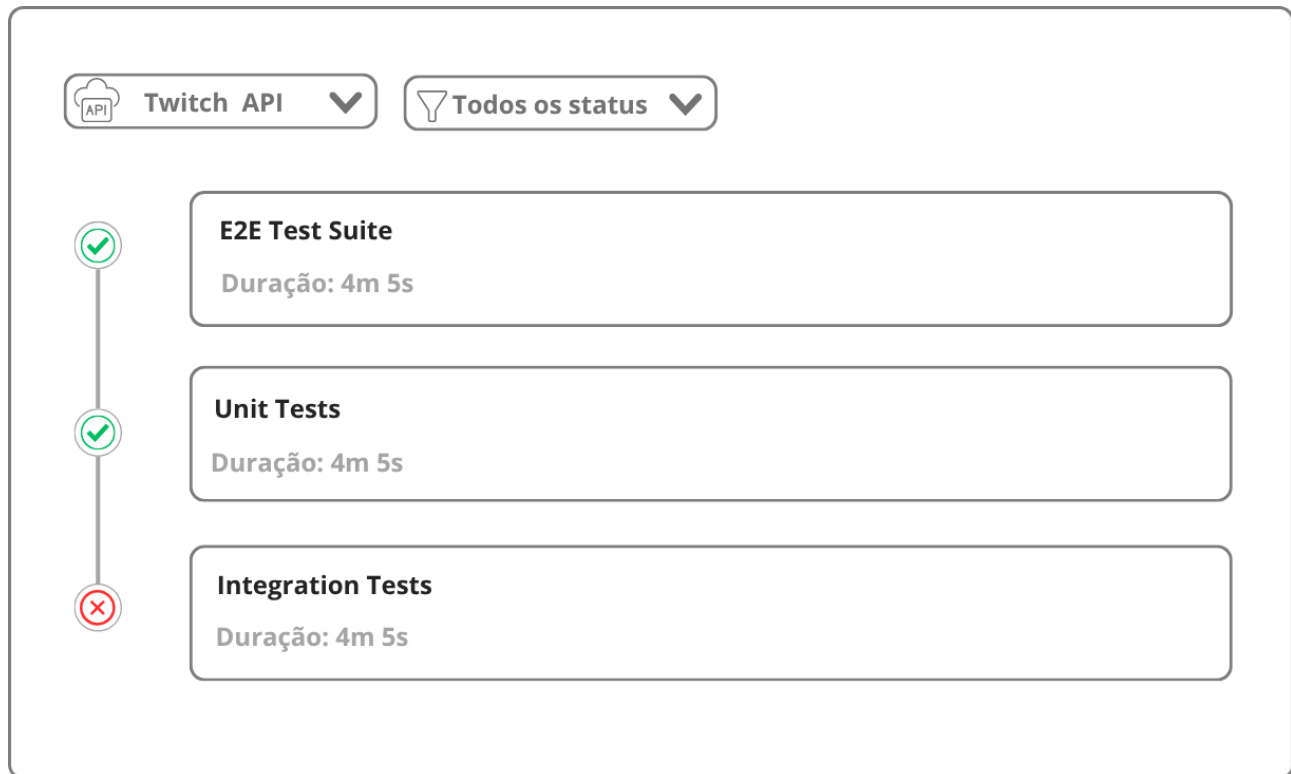


Figura 23: Tela “Histórico de Testes”

Esta tela permite ajustar parâmetros técnicos do sistema, incluindo campos para definir endpoints de APIs, tokens de autenticação e limites de usuários virtuais simultâneos, com um botão “Salvar” para aplicar as alterações, organizada em blocos separados por categoria para priorizar configuração rápida e segura.

**Endpoint da API**
Configure o endpoint da API de videoconferência

URL do Endpoint



**Token de Autenticação**
Configure as credenciais de acesso à API

Token de acesso

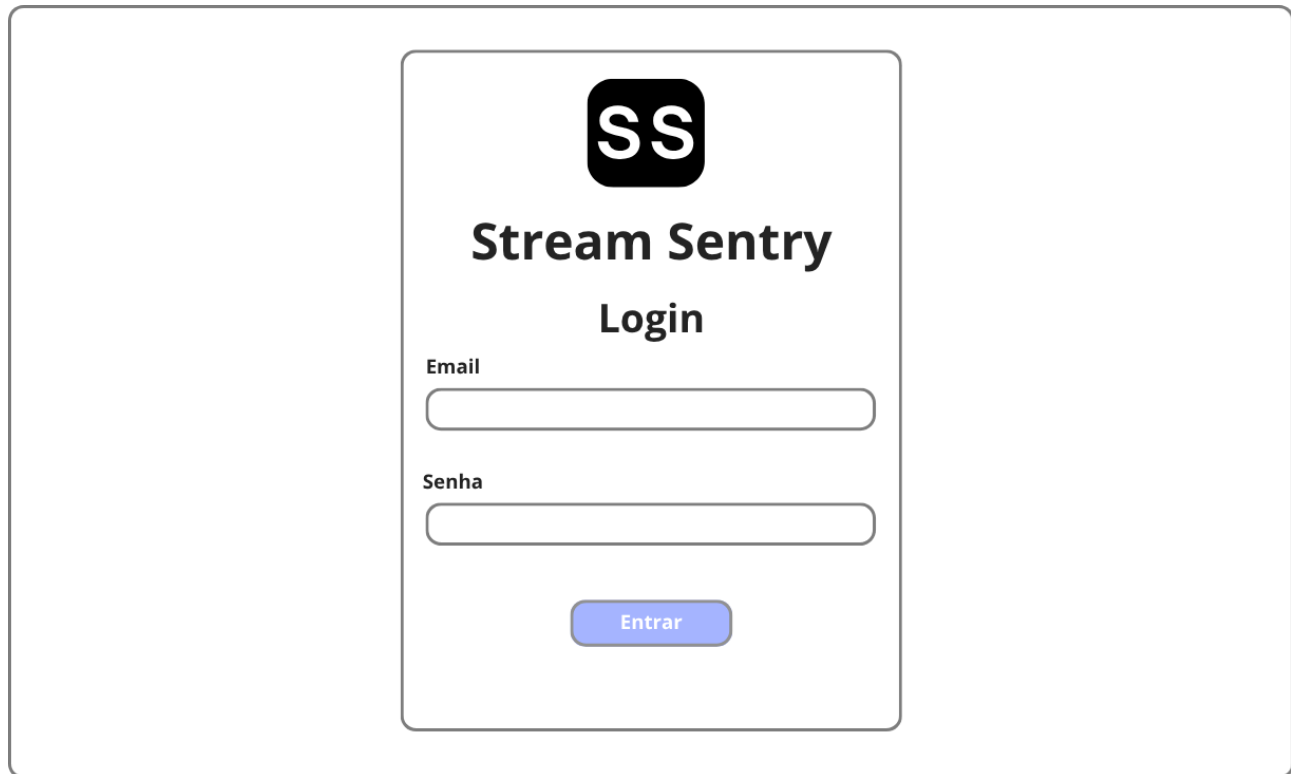
**Usuários Virtuais**
Defina o número máximo de usuários virtuais simultâneos

Limite Máximo	Atual	Em Uso	Disponível
100	100	23	77

 Salvar

Figura 24: Tela “Configurações Avançadas”

Esta tela de login e registro apresenta um design minimalista com fundo neutro, logo centralizada e campos de autenticação (email e senha), e confirmação de senha, com botões “Entrar” em estilo uniforme e feedback visual claro, visando facilitar o acesso rápido ao sistema e manter coerência visual com o restante da aplicação.



The image shows a minimalist login interface for 'Stream Sentry'. It features a central white card on a light gray background. At the top of the card is a black square logo with the white letters 'SS'. Below the logo, the text 'Stream Sentry' is displayed in a bold, black, sans-serif font, followed by 'Login' in a slightly smaller, bold, black font. Underneath, there are two input fields: the first is labeled 'Email' and the second is labeled 'Senha' (Password). Both labels are in a small, gray font. The input fields are white with a thin gray border. At the bottom of the card is a blue button with the white text 'Entrar' (Enter).

Figura 25: Tela “Login e Registro”

5. Modelos de Projeto

A seção de Modelos de Projeto apresenta os artefatos detalhados que sustentam a arquitetura e a implementação do Stream Sentry, garantindo consistência entre o Documento de Visão, os diagramas UML e a base de código. O modelo de dados é relacional e formalizado pelo Diagrama de Entidade e Relacionamento (DER) (Figura 25), composto por quatro entidades principais — Usuario, Teste, Relatorio e ConfiguracaoAPI — com relacionamentos 1:N e 1:1, chaves primárias UUID e tipos ENUM para integridade e eficiência. Esse modelo suporta todas as operações do sistema (configurar, executar, relatar) e será implementado em PostgreSQL com ORM (TypeORM/Prisma), sendo consistente com o Diagrama de Classes (Seção 3.1). O glossário de atributos (Tabela 1) lista todos os campos reconhecidos nas entradas (configuração de testes) e saídas (relatórios), com formato, restrições e descrição clara, facilitando a validação de dados e a comunicação entre frontend, backend e banco. Juntos, esses modelos promovem normalização, manutenibilidade e escalabilidade, atendendo aos requisitos funcionais e não funcionais do sistema.

5.1 Modelo de Dados

O modelo de dados do Stream Sentry é relacional e representado pelo Diagrama de Entidade e Relacionamento (DER) (Figura 16), composto por quatro entidades principais: Usuario (com atributos de autenticação e função), Teste (configuração e execução com status gerenciado por flags), Relatório (resultados e arquivos exportados) e Configuração API (credenciais de integração). Os relacionamentos são 1:N entre Usuario e Teste, 1:1 entre Teste e Relatório, e 1:N entre Usuario e Configuração API. O uso de chaves primárias UUID, chaves estrangeiras e tipos ENUM garante integridade referencial, normalização e eficiência em consultas. O modelo suporta todas as operações do sistema (configurar, executar, relatar), é consistente com o Diagrama de Classes (Seção 3.1) e será implementado em PostgreSQL com ORM (TypeORM/Prisma).



Figura 26: Diagrama de Entidade e Relacionamento

5.2 Glossário

O Glossário de Atributos (Tabela 4) formaliza a definição técnica de todos os campos de dados manipulados pelo sistema Stream Sentry, servindo como uma referência central para o desenvolvimento do schema do banco de dados e para a validação de interfaces. A tabela abrange desde os parâmetros essenciais de configuração dos testes, como `quantidade_usuarios` e `api_tipo`, até as métricas críticas de desempenho coletadas em tempo real, como `latencia_media_ms` e `taxa_falhas_percent`. Para cada atributo, são especificados o formato de dados (incluindo tipos primitivos como INT e FLOAT, bem como ENUMs para controle de estado), as restrições de domínio (como o limite de 1 a 500 usuários virtuais) e a descrição funcional, garantindo a padronização e a integridade referencial das informações que transitam entre o frontend, a API e a camada de persistência.

Atributo	Formato	Descrição
<code>quantidade_usuarios</code>	INT (1–500)	Número de usuários virtuais simulados no teste.
<code>api_tipo</code>	ENUM('WEBRTC','JITSI','ZOOM')	Tipo de API de videoconferência utilizada no teste.
<code>duracao_min</code>	INT (1–1440)	Duração do teste em minutos.
<code>status</code>	ENUM('CONFIGURADO','EXECUTANDO','CONCLUIDO','FALHO')	Estado atual do teste no sistema.
<code>latencia_media_ms</code>	FLOAT	Média de latência de rede durante o teste (em milissegundos).
<code>taxa_falhas_percent</code>	FLOAT (0.0–100.0)	Percentual de falhas de conexão ou pacotes perdidos.
<code>cobertura_testes_percent</code>	FLOAT (0.0–100.0)	Percentual de funcionalidades da API cobertas pelo teste.

Atributo	Formato	Descrição
caminho_arquivo_json	VARCHAR(255)	Caminho ou URL do relatório gerado em formato JSON.
caminho_arquivo_csv	VARCHAR(255)	Caminho ou URL do relatório gerado em formato CSV.
endpoint	VARCHAR(255)	URL base da API de videoconferência.
token	TEXT	Token de autenticação para APIs que exigem (Zoom, Jitsi custom).
max_usuarios_simultaneos	INT	Limite máximo de usuários virtuais por execução.
iniciado_em	TIMESTAMP	Data e hora do início da execução do teste.
concluido_em	TIMESTAMP	Data e hora do término da execução do teste.
gerado_em	TIMESTAMP	Data e hora da geração do relatório.

Tabela 4: Glossário

6. Plano de Testes

Esta seção detalha a estratégia de garantia de qualidade do *Stream Sentry*, dividida em testes de aceitação, focados na validação das necessidades de negócio descritas no Documento de Visão, e testes de integração, focados na comunicação entre os componentes do sistema e serviços externos.

6.1 Plano de Teste de Aceitação

O Plano de Teste de Aceitação tem como objetivo validar se as funcionalidades entregues correspondem às necessidades de negócio definidas no Documento de Visão.

A primeira necessidade abordada refere-se à Configuração Flexível e Automação de Testes, cujo objetivo é garantir a parametrização correta para diferentes provedores de vídeo, respeitando restrições de entrada e segurança. Conforme detalhado na Tabela 5, o cenário TA-01 verifica o fluxo positivo de uma configuração válida no Jitsi para iniciar instâncias virtuais, enquanto o TA-02 aplica testes de fronteira (*Boundary Value*) para impedir que o usuário exceda o limite físico de 500 usuários por instância. Simultaneamente, a segurança é testada no cenário TA-03, que submete credenciais inválidas à API do Zoom para assegurar que o sistema trate adequadamente os erros de autenticação (*Unauthorized*).

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-01	Configuração de Teste Jitsi com Sucesso	Usuário logado; API Jitsi disponível.	API: Jitsi Meet Usuários: 50 Duração: 5 min Ação: Clicar em "Iniciar Teste"	O sistema deve validar os inputs, redirecionar para a tela "Executando", exibir o status "Iniciando Workers" e confirmar a criação de 50 instâncias virtuais.

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-02	Validação de Limite de Usuários (Boundary Value)	Usuário na tela de configuração.	API: WebRTC Native Usuários: 501 (Limite é 500) Duração: 2 min	O botão "Iniciar" deve ser desabilitado ou exibir erro imediato: <i>"O limite máximo de usuários por instância é 500"</i> . O teste não deve iniciar.
TA-03	Tratamento de Credenciais Inválidas (Zoom SDK)	Usuário possui <i>token</i> expirado ou inválido.	API: Zoom SDK Token: eyJhbGciOi... (Inválido) Ação: Clicar em "Iniciar Teste"	O sistema deve tentar a conexão, receber erro 401 Unauthorized da API externa e exibir <i>toast</i> de erro: <i>"Falha na autenticação: Verifique seu Token JWT"</i> .

Tabela 5: Configuração e Automação

A segunda necessidade foca no Monitoramento de Qualidade e Coleta de Métricas, visando assegurar a precisão da telemetria em tempo real e o alerta sobre instabilidades. Para validar esses requisitos (Tabela 6), o teste TA-04 simula picos de latência para confirmar a alteração visual dos indicadores de saúde para o estado crítico. Já o cenário TA-05 verifica a resiliência dos contadores de usuários ativos e falhas diante de perdas de pacotes ou desconexões forçadas. O ciclo de vida correto do teste é garantido pelo caso TA-06, que confirma o encerramento automático (*teardown*) das instâncias Puppeteer e a liberação do relatório exatamente ao término da duração estipulada.

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-04	Visualização de Latência Crítica	Teste em execução.	Simulação: Injeção de latência de rede (500ms) em 20% dos clientes virtuais.	O gráfico de linha "Latência Média" deve apresentar um pico acima de 400ms. O indicador de saúde do sistema deve alterar a cor de Verde para Vermelho (Crítico).
TA-05	Contagem de Falhas de Conexão (Packet Loss)	Teste em execução com 10 usuários.	Simulação: Interrupção forçada da rede em 1 cliente virtual (simulação de queda).	O contador "Usuários Ativos" deve cair para 9 e o contador "Erros de Conexão" deve incrementar em +1. O teste não deve ser abortado para os demais usuários.

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-06	Encerramento Automático e Consistência	Teste configurado para 1 minuto.	Ação: Aguardar o cronômetro atingir 00:00.	O sistema deve forçar o <i>teardown</i> (desconexão) de todos os clientes Puppeteer, alterar o <i>status</i> do teste para "Concluído" e habilitar o botão "Ver Relatório".

Tabela 6: Monitoramento de Qualidade

A terceira necessidade cobre a Análise de Dados e Relatórios, validando a persistência e integridade dos dados históricos para ferramentas externas. Conforme a Tabela 7, o cenário TA-07 assegura que a exportação JSON contenha todas as chaves obrigatórias (*test_id*, *metrics*, *summary*) e dados consistentes. A persistência e ordenação cronológica correta na interface de histórico são verificadas pelo teste TA-08. Adicionalmente, o caso TA-09 valida a funcionalidade de filtragem, garantindo que o usuário consiga isolar visualmente os relatórios na tabela com base no provedor de API utilizado (ex: exibir apenas testes Zoom).

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-07	Integridade da Exportação JSON	Teste concluído com ID #105.	Tela: Detalhes do Relatório Ação: Clicar em "Exportar JSON"	<i>Download</i> do arquivo report_105.json. O arquivo deve ser um JSON válido contendo as chaves: test_id, metrics_array, config, e summary (média de latência e total de falhas).
TA-08	Persistência no Histórico	Teste TA-06 recém-concluído.	Ação: Navegar para a rota /history (Histórico).	O teste mais recente deve aparecer no topo da lista, exibindo corretamente: Data atual, Duração (1 min), API utilizada e Status "Sucesso".

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-09	Filtragem de Relatórios por API	Existem registros de testes Jitsi e Zoom no banco.	Filtro: Selecionar "Zoom SDK" no <i>dropdown</i> de filtros.	A tabela deve recarregar exibindo apenas as linhas onde a coluna "Provedor" é igual a "Zoom". Registros de Jitsi devem ser ocultados.

Tabela 7: Análise e Relatórios

6.2 Plano de Teste de Integração

O teste de integração foca na comunicação entre os módulos internos do Stream Sentry e os serviços externos, assegurando que as unidades de software interajam conforme o esperado. Dada a arquitetura do sistema baseada em microsserviços lógicos, a estratégia adotada será a Incremental Bottom-Up, iniciando a validação pela camada de persistência e lógica de negócios (backend e banco de dados) antes de avançar para a integração com a interface de usuário. Essa abordagem permite isolar falhas de comunicação em níveis mais baixos antes de testar fluxos completos na camada de apresentação.

Para operacionalizar essa estratégia, a execução utiliza contêineres Docker para isolar o Banco de Dados, garantindo um ambiente efêmero e limpo para cada bateria de testes. No que tange às APIs externas (Jitsi e Zoom), serão utilizados *Mocks* e bibliotecas como Nock dentro do pipeline de Integração Contínua (CI) para simular respostas HTTP e WebSocket, evitando custos operacionais e a instabilidade inerente à rede externa. As interfaces críticas e os cenários validados neste processo, incluindo a comunicação REST, transações SQL e canais WebRTC, estão detalhados na Tabela 8 a seguir.

ID	Interface Envolvida	Descrição do Cenário	Validação Técnica
TI-01	I-02 (Backend ↔ DB)	Persistência de Configuração O <i>Controller</i> recebe um POST e deve salvar no PostgreSQL.	Enviar POST para /api/tests. Verificar via <i>query</i> SQL direta (SELECT * FROM tests WHERE id = ...) se o registro foi criado com os dados corretos.
TI-02	I-03 (Engine ↔ Jitsi)	Handshake de Sala de Vídeo O <i>Worker</i> deve conseguir entrar em uma sala Jitsi real ou mockada.	O script do Puppeteer deve identificar o elemento DOM #jitsiConferenceFrame (indicando sucesso no carregamento do iframe da API externa) e retornar <i>status code</i> 200.

ID	Interface Envolvida	Descrição do Cenário	Validação Técnica
TI-03	I-01 (Front ↔ Back)	Atualização de Métricas via Socket O Backend emite eventos de telemetria que o Frontend deve consumir.	Emitir evento <code>metric_update</code> no <i>socket</i> do servidor. Verificar se o estado do componente React (<code>useState</code>) é atualizado e se o gráfico renderiza o novo ponto de dados.
TI-04	I-01 e I-02	Fluxo Completo de Relatório Frontend solicita relatório histórico ao Backend.	Requisição GET <code>/api/reports/:id</code> . O Backend deve realizar o <i>JOIN</i> nas tabelas Test e Metrics, serializar o JSON e o Frontend deve renderizar os dados sem erros de tipagem (<code>undefined</code>).

Tabela 8: Integração e Estratégia

7. Cronograma e Processo de Implementação

Esta seção descreve a abordagem de engenharia de software adotada para a construção do *Stream Sentry*, detalhando a metodologia, as ferramentas de controle, o pipeline de integração contínua (CI/CD) e o planejamento temporal para garantir a entrega do projeto dentro do prazo estipulado.

7.1 Cronograma

O cronograma do TCC II será executado em um modelo Ágil, estruturado em dez Sprints de duas semanas cada, cobrindo o desenvolvimento completo do software e a documentação técnica até o final de junho de 2026. Este planejamento prioriza o objetivo central do TCC: as Sprints iniciais (S1 a S5) focam no Core, Métricas e Integrações com todas as APIs de vídeo. As Sprints seguintes (S6 a S8) dedicam-se à Interface Web e aos Testes Integrados (E2E). Por fim, as Sprints finais (S9 e S10)

são reservadas à Coleta de Métricas Definitivas e à Redação dos Capítulos de Resultados da monografia, garantindo que o produto técnico e os dados para o TCC estejam consolidados no prazo.

Mês	Período (Quinzenas)	Sprint	Foco Principal	Atividades Detalhadas
Fev/26	1ª Quinzenal (1-15)	S1	Setup e Core Base	Configuração do ambiente (Node.js/PostgreSQL). Implementação dos modelos de dados e do TestController.
	2ª Quinzenal (16-29)	S2	Motor de Testes	Implementação do TestEngine e MetricCollector base. Primeira execução funcional do teste com WebRTC Native.
Mar/26	1ª Quinzenal (1-15)	S3	Coleta de Métricas	Finalização do MetricCollector. Implementação das rotinas de coleta de latência e taxa de falhas e persistência no BD.
	2ª Quinzenal (16-31)	S4	Integração de APIs	Implementação da conexão com Zoom SDK e Jitsi SDK . Validação da coleta de métricas nas APIs integradas.
Abr/26	1ª Quinzenal (1-15)	S5	Relatórios RAW e Docs	Implementação do ReportGenerator. Geração do Relatório em formato RAW (JSON/CSV). Finalização do Capítulo 3 (Metodologia).

	2ª Quinzenal (16-30)	S6	Interface Essencial	Desenvolvimento das telas Configurar Teste e Executar Teste (monitoramento em tempo real) no React.
Mai/26	1ª Quinzenal (1-15)	S7	Visualização de Dados	Desenvolvimento das telas Relatórios e Dashboard no React, incluindo visualização de gráficos e exportação de dados.
	2ª Quinzenal (16-31)	S8	Testes Integrados (E2E)	Execução da primeira rodada de Testes Integrados (E2E) completos em todos os fluxos e APIs. Correção de <i>bugs</i> críticos.
Jun/26	1ª Quinzenal (1-15)	S9	Qualidade e Docs Técnica	Segunda rodada de Testes Integrados . Elaboração da Documentação Técnica (Arquitetura, APIs) e do Capítulo 4 (Resultados/Análise V1).
	2ª Quinzenal (16-30)	S10	Fechamento e Resultados Finais	Fechamento da Versão 1.2. Coleta de Métricas Definitivas para o TCC. Finalização do Capítulo 5 (Conclusão e Trabalhos Futuros).

Tabela 9: Cronograma

7.2 Metodologia de Desenvolvimento

O desenvolvimento do *Stream Sentry* adota a metodologia Ágil, utilizando uma adaptação do *framework* Scrum para gerenciar o ciclo de vida do software. O trabalho é organizado em *Sprints* quinzenais, o que permite uma entrega incremental de valor e ajustes rápidos de escopo conforme necessário. A estratégia de construção prioriza inicialmente o desenvolvimento do núcleo de execução de testes (MVP), garantindo a estabilidade do *backend*, para posteriormente expandir o foco para a interface de usuário e os módulos avançados de relatórios.

Para a gestão e rastreamento das tarefas, utiliza-se a ferramenta *GitHub Projects* configurada em formato Kanban. O fluxo de trabalho visualiza o progresso das atividades através de colunas de estado, movendo-se de "A Fazer" (*To Do*) para "Em Progresso", "Em Revisão" e finalmente "Concluído" (*Done*). O controle de versão do código-fonte é realizado através do Git, seguindo rigorosamente o modelo *Feature Branch Workflow*. Neste modelo, cada nova funcionalidade ou correção de *bug* é desenvolvida em um ramo (*branch*) isolado. A integração dessas alterações ao ramo principal (*main*) ocorre exclusivamente por meio de *Pull Requests* (PR), que exigem a revisão obrigatória do código por pares antes da aprovação, garantindo a qualidade e a segurança da base de código.

7.3 Ferramentas e Pipeline de CI/CD

Para assegurar a confiabilidade do *software* e automatizar o processo de entrega, foi estabelecido um pipeline de Integração Contínua e Entrega Contínua (CI/CD) utilizando a plataforma *GitHub Actions*. Este mecanismo é fundamental para impedir que códigos com erros de sintaxe ou falhas lógicas sejam integrados ao ambiente de produção, executando uma bateria de verificações automáticas a cada nova submissão de código ao repositório.

O fluxo do pipeline é executado sequencialmente em quatro estágios principais. Inicialmente, ocorre a análise estática e padronização do código (*Linting*) através das ferramentas ESLint e Prettier, que identificam precocemente erros de sintaxe e desvios de estilo. Em seguida, são executados os testes automatizados, que compreendem testes unitários com Jest para validar a lógica dos serviços isolados e scripts de integração para verificar a conectividade com o banco de dados PostgreSQL. Caso os testes sejam bem-sucedidos, o pipeline avança para a etapa de *Build*, onde o *frontend* (React/Vite) e o *backend* (Node/TypeScript) são compilados para garantir a integridade da construção. Por fim, após a aprovação do *Pull Request* na ramificação principal, o processo de *Deploy* (CD) é disparado automaticamente: a interface web é implantada na plataforma Vercel, enquanto o servidor e seus *workers* são containerizados e enviados para o serviço de hospedagem Render ou registro de containers Docker.